

P01 · Ficha de proposta e aprovação do sistema

Apêndice A do manual · Marco 1 (Fundamentos) · versão 0.3

Disciplina	Testes e Qualidade de Software · Ciência da Computação · Unisagrado 2026
Professor	Prof. Dr. Elvio Gilberto da Silva
Turma e grupo	CC · Grupo 13
Integrantes	Leonardo Valentim, Natasha Teixeira e Udymilla Chagas (composição preliminar)
Data	24/09/2026
Repositório	github.com/LeonardoVDN/P2_TQS_2026_CC_GRUPO13_SABORINO

1. Escolha do grupo

Opção 1, **Sistema Saborino**. A Opção 2 (GreenOps Profiler) foi avaliada e descartada porque não atende à seção 5.3: não tem interface web nem formulários suficientes.

2. Verificação dos requisitos mínimos (seção 5.3)

Requisito mínimo	Saborino	GreenOps
Código-fonte disponível	Sim	Sim
Roda em ambiente local ou de teste	Sim (Docker no macOS e Linux sem Docker, verificados em 24/09/2026)	Sim (linha de comando)
4 ou mais fluxos, com 2 ou mais formulários	Sim (7 fluxos, 5 formulários)	Não
Persistência de dados	Sim (PostgreSQL)	Parcial (arquivos de resultado)
Integração entre componentes	Sim (Vue ↔ API ↔ banco)	Só entre módulos internos
Dados fictícios e testes repetíveis	Sim (seed idempotente)	Sim
Disponível durante o semestre	Sim	Sim
Instruções de instalação	Sim (README do grupo, no repositório)	Sim

3. Opção 1: Sistema Saborino (escolhida)

Nome do sistema	Sistema Saborino: gestão de uma pequena produção de alimentos (release a63a24f)
Origem do sistema	Desenvolvido anteriormente por um integrante, Leonardo Valentim (seção 5.2). Os testes usam somente uma cópia local, com banco próprio e dados fictícios.
Link do repositório	https://github.com/LeonardoVDN/P2_TQS_2026_CC_GRUPO13_SABORINO (público). O código fica em 01_SISTEMA_ALV0/saborino.
Licença/autorização	O código é de autoria do integrante, que autoriza o uso acadêmico e as alterações de TDD. A cópia publicada foi anonimizada, e os desvios da release estão em 01_SISTEMA_ALV0/DESVIOS_DA_RELEASE.md. Os dados são somente fictícios.
Tecnologias principais	Python 3.12, Django 5.2, Django REST Framework, PostgreSQL 15, Vue 3 com Vite, Pinia e Axios, autenticação por sessão (cookie e CSRF), Celery e Redis (só para e-mail)
Como executar	Docker (forma A): <code>docker compose -p p2tqs-saborino sobe banco, API (:8000) e frontend (:5173)</code> , depois roda <code>migrate</code> e o <code>seed</code> fictício. Linux sem Docker (forma B): <code>script ambiente_p2/p2_linux.sh</code> . Os passos completos estão no README da pasta 00.
Fluxos selecionados (preliminar)	• F1 Autenticação: login e logout por sessão • F2 Cadastro de clientes: nome, canal, contato e forma de pagamento • F3 Cadastro de produtos: preço, custo, categoria e unidade • F4 Registro de venda: itens, cálculo do total e geração do recebível • F5 Recebimento: pagamento parcial ou total que abate recebíveis • F6 Compras e produção: lançamento de insumos com rateio por produto • F7 Relatórios: painel e fechamento mensal
Integrações existentes	Frontend Vue ↔ API REST (Axios, JSON, sessão e CSRF); API ↔ PostgreSQL (ORM do Django); integração entre módulos (Venda → Recebível → Pagamento/Alocação → Relatórios); API ↔ Celery/Redis ↔ provedor de e-mail (fora do escopo)
Justificativa da escolha	Atende a todos os itens da seção 5.3, com 7 fluxos e 5 formulários. O grupo tem acesso total ao código e conhece o domínio. As regras de cálculo (totais, saldos, status) servem bem para valor limite e partição de equivalência. A API REST permite usar o Postman, e a interface web permite usar o Selenium. Já existe uma suíte de 69 testes para comparar com os testes do grupo.
Riscos de viabilidade	• Risco de afetar dados reais: mitigado com uma cópia local isolada, banco próprio e dados fictícios.

- O e-mail depende de Celery, Redis e um provedor externo: esses fluxos ficam fora do escopo.
- O sistema é maior que o necessário: o escopo fica restrito aos fluxos F1 a F7.
- A linguagem não é Java: pedimos autorização para usar pytest no lugar do JUnit (seção 8.3).

Decisão do professor Aprovado / Aprovado com ajustes / Não aprovado

Ajustes solicitados

Data da aprovação

4. Classificação preliminar dos níveis de teste

Nível / tipo	Finalidade no Saborino	Ferramenta prevista
Unitário	Regras isoladas: arredondamento monetário, subtotal do item, total da venda, status do recebível (Aberto, Parcial, Pago, Vencido) e competência derivada da data	pytest e pytest-django
Integração / API	Cliente ↔ API ↔ banco: autenticação, cadastro de clientes e produtos, venda gerando recebível, pagamento abatendo saldo, respostas 2xx, 4xx e 401	Postman e Newman
Sistema / interface	Fluxos ponta a ponta pela interface: login, cadastro de cliente, venda rápida e recebimento	Selenium WebDriver (Python)
Aceitação	4 cenários do ponto de vista do usuário gestor, executados por uma pessoa externa	Roteiro e registro (Aceito, Aceito com ressalvas ou Não aceito)
Não funcionais	Desempenho básico, compatibilidade (2 navegadores e resoluções), segurança básica e confiabilidade (reinício controlado)	Scripts de medição, Selenium e navegador

Justificativa da estratégia: as regras de cálculo concentram o maior risco de erro silencioso (valores e saldos errados). Por isso, a base da pirâmide são os testes unitários dessas regras. A API é a fronteira principal entre os componentes e recebe os testes de integração. Os fluxos que o usuário mais repete (venda e recebimento) recebem os testes de sistema e de aceitação.

5. Opção 2: GreenOps Profiler (descartada)

Nome do sistema GreenOps Profiler: protótipo que compara o consumo de energia entre versões de funções de backend

Origem do sistema Sistema acadêmico em desenvolvimento pelo grupo (TCC dos mesmos

	integrantes)
Link do repositório	Não publicado
Licença/autorização	Código de autoria do grupo
Tecnologias principais	Python 3.11+, numpy, scipy e pytest. Há um backend Django de apoio, usado só como alvo das medições.
Como executar	Linha de comando (<code>cd sandbox; ./run.sh</code>). Não tem interface web para o usuário final.
Fluxos selecionados (preliminar)	Comparação de variantes pela linha de comando, autoteste A/A e geração de relatório em texto
Integrações existentes	Entre módulos internos (runner, stats, report) e com a fonte de energia (RAPL ou simulada)
Justificativa da escolha	O grupo domina o código, e ele tem muita lógica testável por unidade.
Riscos de viabilidade	- Não atende à seção 5.3: faltam interface web e ao menos 2 formulários, o que inviabiliza Selenium e Postman. - A medição por RAPL não existe no hardware disponível (Apple Silicon). - Mistura a avaliação da P2 com o TCC.
Decisão do professor	Aprovado / Aprovado com ajustes / Não aprovado
Ajustes solicitados	
Data da aprovação	

6. Evidências anexas

Execuções reais, com a saída gravada em 01_SISTEMA_ALV0/evidencias_ambiente/:

Arquivo	Resultado
2026-09-24_suite_existente_django.txt	69 testes do backend, OK (Linux, PostgreSQL 15.18)
2026-09-24_smoke_api.txt	Login (204/401), clientes (200/201/400), painel (200) e acesso sem login (401)
2026-09-24_frontend_vitest_build.txt	Vitest (1 teste, OK) e build do Vue concluído
2026-09-24_docker_migrate_seed.txt	Docker no macOS: 45 migrações aplicadas e seed fictício (3 clientes, 3 produtos, 2 vendas)
2026-09-24_docker_validacao_codigo_anonimizado.txt	Após a anonimização: 69 testes OK, smoke com os mesmos 9 resultados, Vitest e build OK

Prof. Dr. Elvio Gilberto da Silva — Decisão sobre a proposta

Bauru, _____ de _____ de _____

P2_TQS_2026_CC_GRUPO13_SABORINO

Leonardo Valentim · Natasha Teixeira · Udyrilla Chagas ·
github.com/LeonardoVDN/P2_TQS_2026_CC_GRUPO13_SABORINO